CS 577 Deep Learning
Homework 1: Tools and Tensor Fluency

Your Name (Hawk username: `Irma Modzgvrishvili`)

Fall 2026

Part 1: LaTeX and Mathematical Communication

1.1 Tensor notation

Typest the following statement cleanly, including dimensions:

> For a batch X with shape (B, D), weights W with shape (D, K), and bias b with shape $(K,)$, the affine output is
>
> $$Y = XW + 1_B b^T$$
>
> and has shape (B, K)

Here, $1_B \in \mathbb{R}^{B\times 1}$ denotes the all-ones column vector. Therefore, $1_B b^T \in \mathbb{R}^{B\times K}$ repeats the bias vector b^T once for each of the B examples. This is the mathematical form of broadcasting b across the batch dimension in the NumPy/PyTorch expression

$$Y = XW + b$$

Use mathematical notation rather than copying the monospace statement verbatim.

1.2 Regularized mean-squared error

Typest the objective

$$J(w) = \frac{1}{N}\|Xw - y\|_2^2 + \frac{\lambda}{2}\|w\|_2^2$$

and its gradient with respect to w

$$\nabla_w J(w) = \frac{2}{N}X^T(Xw - y) + \lambda w$$

1.3 Results table and interpretation

Quantity	Value
Initial training loss	2.347500
Final training loss	1.257789e-06
Learned slope	2.498237
Learned intercept	-0.399999
Maximum gradient difference	3.100408818568212e-12

Table 1: Results from the deterministic regression and gradient checks.

Interpretation: The model optimization run successfully because training loss drop from initial value 2.3475 down to final loss 1.25779. The algorithm learn a slope around 2.49823 and intercept around -0.39999 to fit target dataset. Also, the maximum gradient difference between numeric check and math formula is extremely small. This confirm that our analytical gradient derivation is implemented correct.

Part 2: NumPy Tensors and Vectorization

2.1 Shapes and broadcasting

$$X \in \mathbb{R}^{4\times 3}, \qquad W \in \mathbb{R}^{3\times 2}, \qquad b \in \mathbb{R}^{2}, \qquad Y \in \mathbb{R}^{4\times 2}.$$

$X@W$ is matrix multiplication. Since X has shape $(4, 3)$ and W has shape $(3, 2)$, the inner dimensions match $(3 = 3)$, so the result has shape $(4, 2)$. The bias b has shape $(2,)$. Broadcasting compares dimensions from right to left, so $(2,)$ is treated like $(1, 2)$. Since $2 = 2$ and 1 can expand to 4, b is broadcast to shape $(4, 2)$ and added to every row of $X@W$.

$$Y = \begin{bmatrix} 4.25 & -3.75 \\ 0.25 & 5.25 \\ -2.25 & -0.75 \\ -1.25 & 4.25 \end{bmatrix}$$

2.2 Standardization

Mean:

$$[0.5,\ 0.25,\ 1.25]$$

Standard deviation:

$$[1.11803399,\ 1.47901995,\ 1.47901995]$$

check mean:

$$[0.0,\ -1.38777878 \times 10^{-17},\ 0.0]$$

check std:

$$[1,\ 1,\ 1]$$

2.3 Pairwise squared distances

$$D^2 = \begin{bmatrix} 2 & 1 \\ 1 & 4 \\ 2 & 5 \end{bmatrix}$$

The inserted singleton dimensions change the shapes as follows:

$$A[:, \text{None}, :] \to (3, 1, 2), \qquad B[\text{None}, :, :] \to (1, 2, 2).$$

Broadcasting then expands these arrays to a common shape of

$$(3, 2, 2),$$

so that every row of A is compared with every row of B simultaneously. After squaring the elementwise differences and summing over the last dimension, the final pairwise squared-distance matrix has shape

$$(3, 2).$$

2.4 Vectorization timing

The benchmark used random arrays with shapes

$$A_{\text{bench}} \in \mathbb{R}^{200\times16}, \qquad B_{\text{bench}} \in \mathbb{R}^{250\times16}.$$

The loop-based and vectorized implementations were timed using `timeit.repeat` with five repetitions, and the median runtime was reported.

$$\text{Loop median} = 0.0961331 \text{ s}$$

$$\text{Vectorized median} = 0.00216217 \text{ s}$$

$$\text{Local speed ratio} = \frac{0.0961331}{0.00216217} \approx 44.46.$$

In this local benchmark, the vectorized implementation was approximately 44.5× faster than the explicit Python-loop implementation. This result reflects the tested array sizes and the local execution environment, so it should not be interpreted as a universal speedup for every machine or input size.

Part 3: PyTorch Tensors and a Minimal Model

3.1–3.2 Agreement, dtype, and device

The maximum absolute difference between the NumPy and PyTorch affine outputs was

$$0.0.$$

The tensors were created using `torch.float64` and placed on the selected device. Tensors participating in the same operation generally need to be on the same device because CPU and accelerator memory are separate. Their dtypes should also be compatible; otherwise PyTorch may raise an error or perform an unintended type conversion.

3.3–3.4 Model and training

The model uses one `nn.Linear(1,1)` layer, mapping an input of shape $(B, 1)$ to an output of shape $(B, 1)$.

The weight and bias were initialized to zero. The model was trained for 100 full-batch steps using mean-squared error and SGD with learning rate 0.1.

- Initial loss: 2.3475000858
- Final loss: 1.2577893×10^{-6}
- Learned weight: 2.4982371330
- Learned bias: -0.3999999762

Selected loss values were:

Step	MSE Loss
0	2.3475000858
9	0.5953179002
49	0.0017835345
99	1.2577893×10^{-6}

Part 4: Three Views of the Same Gradient

4.1 Analytic gradient

The regularized mean-squared error objective is

$$J(w) = \frac{1}{N}\|Xw - y\|_2^2 + \frac{\lambda}{2}\|w\|_2^2.$$

We differentiate the two terms separately. For the mean-squared error term,

$$\nabla_w \left(\frac{1}{N}\|Xw - y\|_2^2 \right) = \frac{2}{N} X^T (Xw - y).$$

For the L_2 regularization term,

$$\nabla_w \left(\frac{\lambda}{2}\|w\|_2^2 \right) = \lambda w.$$

Therefore, the full gradient is

$$\boxed{\nabla_w J(w) = \frac{2}{N} X^T (Xw - y) + \lambda w}.$$

This is the analytic gradient implemented in `mse_l2_grad_numpy`.

4.2–4.3 Numerical comparison

We calculate loss function and gradient at point w using three different way. The analytic gradient and PyTorch autograd match perfectly (0.0 difference) cuz autograd calculate exact derivatives using chain rule. The finite difference method has a super small error ($\sim 10^{-12}$) cuz it is a numerical approximation using small step size ϵ. This tiny difference proves our math code and autograd implementation are both correct.

Metric	Value / Result
Objective value	0.2031666666666667
Analytic gradient	[-0.88, -0.16333333]
Finite difference gradient	[-0.88, -0.16333333]
Autograd gradient	[-0.88, -0.16333333]
Max Absolute Pairwise Differences:	
Analytic vs Finite difference	3.100408818568212e-12
Analytic vs Autograd	0.0
Finite difference vs Autograd	3.100408818568212e-12

Part 5: Reflection

Shape/broadcasting mistake. When doing squared distance, forgetting to use A[:, None, :] and B[None, :, :] break the shapes and broadcasting doesnt work right.

Gradient validation method. Using finite difference formula

$$\frac{f(w+\epsilon) - f(w-\epsilon)}{2\epsilon}$$

gives us a easy way to check if the math gradient and PyTorch autograd are correct.

Workflow change before HW2. Before HW2, I will always run unittest script first to catch small bugs early.

Assistance and Generative-AI Disclosure

I used YouTube tutorials to better understand how nn.Module works, including how different modules and functions can be organized within the same codebase. I used LaTeX documentation and cheat sheets to help format and structure the written report. I also used grammar tools to improve sentence clarity and correctness. I verified the code behavior and mathematical results myself and made the final implementation and wording choices independently.